BANTAY-BAHA CODE GUIDE

A walkthrough of every file and every code block in the repository: what it does, how, and why it was written that way. Line links point at the code as of 2026-10-24. `python scripts/check_code_guide.py` checks every link and every function row against the code (section 7), so run it after changing code and fix what it lists.

How to read this. Section 1 is the big picture (read it first). Sections 2-6 follow the data from download to dashboard, one file at a time, in the order the pipeline runs. Sections 7-8 cover the documents and the optional benchmark models. Each entry is one function or one top-level block.

CONTENTS

1. The big picture
2. Entry points (.bat files)
3. Getting the data: `scripts/download_data.py` , `download_gee.py` , `sync_data.py`
4. The `bantay` library
   - 4.1 config · 4.2 units · 4.3 dengue · 4.4 storms · 4.5 rain · 4.6 rivers · 4.7 grids · 4.8 flood · 4.9 panel · 4.10 leadlag · 4.11 forecast · 4.12 access · 4.13 scenarios · 4.14 lepto
5. The build: `scripts/build.py`
6. The dashboard: `app/streamlit_app.py`
   - 6.1 ui · 6.2 sus · 6.3 lepto_tab · 6.4 theme · 6.5 tests
7. Documents: `make_results.py` , `results_pages.py` , `results_slides.py` , `make_proposal.py` , `check_code_guide.py`
8. Benchmark models: `scripts/benchmark_*.py`
9. The EDA notebook
10. Every file the pipeline writes
11. Glossary

---

1. THE BIG PICTURE

Bantay-Baha answers two questions for a typhoon track: **where will dengue surge** (R) and **who loses their route to a hospital** (A). Priority is **P = R x A**, and simple rules turn R and A into an action tier per province.

```
download_data.py / download_gee.py / sync_data.py  ->  data/raw/           (21+ sources, MANIFEST.csv)
                                                          |
scripts/build.py                                          v
  crosswalk  units.py       126 dengue units -> 85 harmonised provinces       (data/crosswalks/, units.gpkg)
  panel      panel.py       one row per province per week: cases, rain, rivers, storms, flood     (panel.parquet)
               uses dengue.py, storms.py, rain.py, rivers.py, grids.py, flood.py
  leadlag    leadlag.py     PCMCI+ lead-lag graph, sentinels, communities
  forecast   forecast.py    quantile XGBoost + ablations, rolling-origin test, WIS, calibrated surge risk R
  ensemble   forecast.py    re-score with any bench_<model>.parquet + their median ensemble (no refit)
  scenarios  scenarios.py   4 typhoons x 15 tracks: R (forecast) x A (access.py: network + raster) -> P, tiers
  catalog    scenarios.py   observed track of every other 2016-2020 storm whose gale swath reaches land
  app        (simplified map)
  lepto      lepto.py       leptospirosis lag analysis (national + 12 cities), typhoon event study
                                                          |
                                                          v   data/processed/app/
app/streamlit_app.py      reads data/processed/app/ only     dashboard (with app/ui.py, sus.py, lepto_tab.py)
scripts/benchmark_*.py    separate GPU venv: bench_<model>.parquet  ->  build.py ensemble
scripts/make_results.py   reads data/processed/           ->  docs/results/ (report, slides, demo script, this guide)
```

`python scripts/build.py` runs all stages in this order (`crosswalk panel leadlag forecast ensemble scenarios catalog app lepto`); `build.py panel forecast` runs only those. Four rules shape the whole codebase:

- **Missing data never stops a build.** Every stage checks its inputs with `config.have()` / `config.missing()`; missing inputs leave columns empty (NaN) and are listed in `data/processed/status.json`. Re-running after the files arrive picks them up with no code change. Nothing is ever filled with fake numbers.
- **No look-ahead.** Forecasts are tested by rolling origin (train on the past, predict the future, move forward); the graph, the surge thresholds and the calibration of R are re-learned inside each training window. A Sentinel-1 flood map is used only from the week its last image exists (landfall + `S1_AFTER_DAYS`).
- **Heavy work happens in the build, not the dashboard.** The dashboard only reads small pre-computed files, so it answers in seconds.
- **Optional outputs slot in.** The dashboard and the documents show a section only when its file exists, so a partial build still renders.

2. ENTRY POINTS (.BAT FILES)

FILE	WHAT IT DOES
`setup.bat`	Team setup: creates the venv at `%USERPROFILE%\.venvs\bantay-baha` (outside OneDrive so it isn't synced), installs `requirements.txt`, registers the Jupyter kernel "Python (bantay-baha)", signs in to Google once (`earthengine authenticate`) if there are no saved credentials, then runs `scripts/sync_data.py pull` to copy `data/raw/` from the team bucket (source names given to `setup.bat` are passed on). Each step is skipped if already done.
`download_data.bat`	The slow path: same venv setup, then downloads every source from its original website (`download_data.py`) and, if a GEE project id is given, the Earth Engine sources (`download_gee.py`). Ends by listing the two manual downloads (PSGC, NHFR). Used to (re)build the bucket.
`build.bat`	Runs `scripts/build.py` with the venv's Python, passing any stage names through (`build.bat panel forecast`).
`run_app.bat`	Starts the dashboard from the repo folder (so Streamlit picks up `.streamlit/config.toml`): `python -m streamlit run app\streamlit_app.py`.

3. GETTING THE DATA

scripts/download_data.py (OPEN SOURCES)

Downloads every open dataset into `data/raw/<source>/` and records each file in `data/raw/MANIFEST.csv` (source, path, bytes, sha256, url). Safe to re-run: finished files are skipped.

BLOCK	LINES	EXPLANATION
Constants `ROOT`, `RAW`, `UA`, `HDX`	23-26	Repo root and raw folder found from the script's own location (works from any working directory). `UA` is a polite User-Agent; some servers reject Python's default.
`STATIC`	29-90	The catalogue of single-file sources: `source -> [(url, filename)]`. Adding a source = adding a line here. HydroRIVERS uses the Australasia (`au`) file because the Philippines is not in the Asia file.
`EMS`, `NOAH_FOLDERS`, `NOAH_DOCS`	92-95	Sources that need scraping: Copernicus EMS activations (with a filename filter) and the UP NOAH Google Drive folder and document ids.
`CHIRPS_BBOX`, `CHIRPS_YEARS`	98-99	The Philippine bounding box and years (2016-2020) for the server-side CHIRPS subset.
`PARTIAL`	101	Collects single files that failed inside a source that otherwise finished, so one bad file doesn't abort a whole source.

BLOCK	LINES	EXPLANATION
`fetch()`	104-125	The download primitive. Skips existing non-empty files; streams to `name.part` and renames only when complete (an interrupted run never leaves a half file that looks finished); retries 3 times with back-off; with `binary=True` refuses an HTML page (a quota or login page) masquerading as a file.
`drive_listing()`	128-130	Lists a public Google Drive folder by parsing its embedded-view HTML (no API key needed).
`fetch_drive()`	133-135	Downloads a Drive file via `drive.usercontent.google.com` with `confirm=t`, which skips the virus-scan page Drive shows for files over 100 MB.
`pull_static()`	138-139	Fetches every `(url, filename)` of one `STATIC` source.
`pull_ems()`	142-149	Scrapes each EMS activation page for `*_vector.zip` links and downloads the ones matching the filter (EMSR312 delineation packages; all of EMSR478).
`pull_noah()`	152-165	Downloads the NOAH readme, metadata and licence, then every province zip in the 5-yr and 25-yr folders; a failed province is recorded in `PARTIAL` and the rest continue.
`pull_chirps()`	168-176	Builds one IRI Data Library URL per year that subsets CHIRPS daily rainfall to the Philippine box on the server, then downloads it as NetCDF.
`PULLERS`	179	Maps every source name to its puller, so `main()` can run all of them or a chosen few.
`sha256()`	182-187	Checksum of a file, read in 1 MB chunks (works for multi-GB files).
`write_manifest()`	190-205	Merges new rows into `MANIFEST.csv`; recomputes the checksum only when a file's size changed (avoids re-hashing 7 GB on every run).
`main()`	208-226	Runs the selected pullers; writes the manifest after each source (an interrupted run keeps what finished); prints failures and exits non-zero if anything failed.
`if __name__ == "__main__"`	229-230	Command-line entry: source names are optional arguments.

scripts/download_gee.py (GOOGLE EARTH ENGINE SOURCES)

Needs a registered Earth Engine project (`--project <id>`). Pulls IMERG rainfall, the MAP friction surface, Global Flood Database events, a Sentinel-1 scene inventory and the Sentinel-1 flood masks. Like `download_data.py`, every file goes to `<name>.part` first and is renamed when complete, and each source is added to the manifest as soon as it finishes.

BLOCK	LINES	EXPLANATION
Imports, `ROOT`, `RAW`, `BBOX`	20-36	Same layout as `download_data.py`; imports its `write_manifest` so GEE files land in the same manifest.
`TYPHOONS`, `IBTRACS_NAMES`	39-46	The four backtest storms: landfall dates (for before/after windows) and their IBTrACS names (to find the track).
`S1_BEFORE`, `S1_AFTER`	49-50	The Sentinel-1 windows in days from landfall: before = days -30 to -3, after = days 0 to +10. `config.S1_AFTER_DAYS` must equal the end of the after window, because a flood map only exists once its last image does (section 4.9).
`done()`	53-58	True (and prints `skip`) if the file is already on disk and not empty; otherwise creates its folder. Every puller calls it first, so re-runs skip finished files.
`save()`	61-64	Writes bytes to `<name>.part`, then renames: an interrupted run never leaves a half file that a later run would skip.
`write_csv()`	67-72	Writes a table (list of dicts) as CSV through `save()`.
`download_image()`	75-95	Asks Earth Engine for a GeoTIFF of an image over a region at a scale and downloads it. EE computes the image on request and sometimes answers 503/429, so it retries up to 4 times with a fresh URL and a growing pause. Unzips when EE returns a zip. Skips files already on disk.
`pull_imerg()`	98-114	For each month 2016-2020, turns half-hourly IMERG (mm/h) into daily totals (mean x 24) and stacks the days as bands of one GeoTIFF per month.

BLOCK	LINES	EXPLANATION
`pull_map_friction()`	117-123	Downloads the Malaria Atlas Project 2019 friction surfaces (motorised and walking), about 1 km.
`pull_gfd()`	126-135	Writes a table of Global Flood Database events that intersect the Philippines (dates, cause, deaths). Context only: its events stop in 2017.
`pull_sentinel1()`	138-156	Lists every Sentinel-1 IW VV scene from 24 days before to 14 days after each landfall (no imagery), used to plan the flood mapping.
`track_tiles()`	159-174	Picks the 1-degree tiles of the Philippine box whose centre is within ~350 km (plus half a tile) of a storm's track (reads the local IBTrACS file), so we only process tiles the storm could have flooded.
`s1_flood_image()`	177-212	**The flood-mapping recipe**, computed on Google's servers. For each relative orbit (same satellite geometry) that imaged the tile both before and after landfall: before = median of the smoothed VH backscatter over the `S1_BEFORE` days, after = darkest smoothed image of the `S1_AFTER` days; a pixel is new flood when VH drops by at least 3 dB and ends at or below -18 dB (water is radar-dark). Floods from all orbits are combined; permanent water (JRC), slopes over 5 degrees and specks under 8 pixels are removed. Output: 1 = new flood, 0 = dry, 255 = no data or sea. *Why not plain UN-SPIDER?* Its after/before ratio on dB values under-detects surfaces that were already dark: at Tuguegarao it caught 2 of 12 EMS-flooded sample points, this rule 6 of 12.
`STORM_FILTER`, `pull_s1_flood()`	215-237	Loops over storms (or only those given with `--storms`) and their tiles; builds the flood image per tile and downloads it at 100 m. Tiles no orbit covered are reported as `none` and skipped.
`PULLERS`, main block	240-254	Command-line entry: initialises Earth Engine with the project, runs the chosen pullers and writes the manifest per source.

scripts/sync_data.py (TEAM BUCKET)

Mirrors `data/raw/` to and from the private Cloud Storage bucket, so teammates don't redo slow downloads.

BLOCK	LINES	EXPLANATION
`PROJECT`, `BUCKET`, `LOCATION`, `PREFIX`	25-28	Where the team's copy lives (Singapore region).
`client()`	31-39	Makes a Storage client from Application Default Credentials, falling back to the Earth Engine login.
`read_manifest()`, `write_manifest()`	42-50	Read/write `MANIFEST.csv` as a dict keyed by path.
`push()`	53-76	Creates the bucket if needed (private, uniform access), uploads every manifest file the bucket lacks or has a different checksum for, then uploads the manifest itself.
`pull()`	79-102	Downloads every file in the bucket's manifest that is missing locally, to `.part`, checks its sha256 and only then renames it; merges the rows into the local manifest.
main block	105-112	`python scripts/sync_data.py pull|push [sources...]`.

4. THE bantay LIBRARY

`bantay/__init__.py` only holds the package docstring. Every module below is imported by `scripts/build.py`, and some by the dashboard, the benchmarks and the results scripts.

4.1 bantay/config.py

Shared paths, constants and the missing-input registry.

BLOCK	LINES	EXPLANATION
Paths `ROOT` ... `APP_DATA`	12-17	Every folder the pipeline uses, built from the package location: raw inputs, cached intermediates (`interim`), outputs (`processed`), reviewed crosswalks, and the small files the dashboard reads (`processed/app`).
`BBOX` , `CRS_M`	19-20	The Philippine bounding box (lon/lat) and the metric projection used for distances and buffers (UTM 51N).
`WEEK_FREQ` , `FIRST_WEEK` , `LAST_WEEK` , `MODEL_END`	23-25	DOH weeks end on Sunday; the panel spans 2016-01-10 to 2021-01-10. `MODEL_END` (2019-12-29) is the last week any model trains on or is scored on, because 2020 reporting collapsed under COVID-19.
`STORMS`	28-33	The four backtest typhoons: local name, international name, season, landfall date and the folder name used by the GEE downloads.
`S1_AFTER_DAYS`	37	10: a Sentinel-1 flood map uses images up to 10 days after landfall, so it is known only from the DOH week containing landfall + 10 days. Features must not use it earlier. Must match `S1_AFTER` in `download_gee.py` .
`QUANTILES`	40	The 23 quantile levels of the CDC FluSight forecasting hub (0.01, 0.025, 0.05, 0.10 ... 0.95, 0.975, 0.99).
`week_ending()`	43-49	Maps any date (or column of dates) to the Sunday that ends its DOH week. Used whenever daily rain or storm times are joined to weekly cases.
`_STATUS_FILE` , `_load_status()`	54-60	`data/processed/status.json` holds one line per input: available or not, a detail, and when it was checked.
`record()`	63-68	Writes or updates one input's status.
`status()`	71-72	Returns the whole status dict (the build prints it at the end; the dashboard shows it on *Data & caveats*).
`missing()`	75-78	Prints a warning and records the input as missing; returns `None` so callers can write `return missing(...)` .
`have()`	81-90	Checks that paths exist (and, for a folder, that it holds a matching file), records the result, returns True/False.
`ensure_dirs()`	93-95	Creates the output folders.

4.2 bantay/units.py

Builds the 85 **harmonised 2016-2020 provinces** and the crosswalks into them. Needed because the dengue file has 126 reporting units (provinces mixed with cities) and today's boundary file (COD-AB) uses post-2020 boundaries.

BLOCK	LINES	EXPLANATION
Paths	20-23	The COD-AB geodatabase, the two crosswalk CSVs, and the unit polygons output.
`MERGE`	26-27	Undoes post-2020 changes: Maguindanao del Norte/Sur back to Maguindanao (split 2022), the Special Geographic Area back to Cotabato (created 2024 from Cotabato barangays), City of Isabela into Basilan.
`SYNTHETIC_ID`	29	Maguindanao has no single current code, so it gets a clearly synthetic id.
`DENGUE_FIX` , `DENGUE_CITY_PROVINCE`	32-34	Spelling and old-unit fixes for the dengue file (NUEVA VISCAYA, SISQUIJOR, NORTH COTABATO, BASILIG = Bislig; MAGUINDANAO via Maguindanao del Norte; MANILA = NCR First District) and the one ambiguous city (NAGA CITY is in Camarines Sur, from its Region V label).
`norm()`	37-41	Normalises a place name for matching: strips accents (Las Piñas -> LASPINAS), parentheses, "City of"/"City" and anything not a letter.
`read_adm()`	44-48	Reads one COD-AB level (1 = region, 2 = province, 3 = municipality) straight from the zip; records COD-AB as missing (and returns None) if the zip is absent.

BLOCK	LINES	EXPLANATION
`_unit_name()`	51-55	Gives an ADM2 name its harmonised unit name and the rule used.
`build_adm2_crosswalk()`	58-65	ADM2 -> unit table. A unit's id is the official code of the member that shares its name, or the synthetic id.
`build_dengue_crosswalk()`	68-92	Matches each dengue unit to COD-AB: city units are looked up as cities first (so QUEZON CITY is not Quezon province), then province names, then municipality names; each match is folded into its unit. Ambiguous matches get a note.
`write_csv_with_header()`	95-100	Writes a CSV preceded by `#` comment lines that document its source and rules (the repo rule for hand-reviewed tables).
`read_crosswalk()`	103-104	Reads a crosswalk, skipping the comment lines, as text (codes keep leading zeros).
`build_crosswalks()`	107-137	Builds both crosswalks once and reuses them afterwards (unless `--force`), because they were reviewed by hand. Returns `None, None` when COD-AB or the dengue file is missing. Records how many dengue units matched.
`check_psgc()`	140-154	If the PSA PSGC Excel is downloaded, checks every COD-AB province code against it. Optional; skipped when absent.
`build_units_geometry()`	157-172	Dissolves ADM2 polygons into the 85 unit polygons, repairs invalid geometry, simplifies to ~50 m (small enough to process fast, finer than the 100 m rasters) and saves `units.gpkg`.
`load_units()`	175-178	Loads the unit polygons, building them if needed.
`adm3_with_units()`	181-187	Municipality polygons with their unit, for the municipal access results, the municipality raster and Sentinel-1 per municipality.

4.3 bantay/dengue.py

The forecast target and the population used to turn cases into incidence.

BLOCK	LINES	EXPLANATION
`DENGUE`, `CENSUS`	12-13	The two input files.
`weeks()`	16-17	The Sundays from 2016-01-10 to 2021-01-10 (262 weeks).
`load_dengue()`	20-28	Reads the DOH file, skipping row 2 (HXL tags), stripping stray spaces from names, checking that every date is a Sunday, and dropping `deaths` (2016 deaths are implausible).
`unit_week_cases()`	31-45	Builds the full unit x week grid: sums the member reporting units of each harmonised unit and keeps the total **only if every member reported** that week, so a missing city never looks like zero cases.
`unit_population()`	48-77	2020 census population per unit. The census's municipal codes are the old 9-digit PSGC and don't join to COD-AB for cities or post-2022 Maguindanao, but its *Province* column already places cities in their province, so units are matched on that column by name, with fixes for the NCR districts, SGA and City of Isabela.

4.4 bantay/storms.py

Typhoon tracks from IBTrACS: wind radii, swaths, shifted tracks and weekly exposure.

BLOCK	LINES	EXPLANATION
`NM_KM`, `IBTRACS`, `R34`, `COLS`	18-22	Nautical miles to km, the file, the four 34-knot radius columns, and the columns read.
`load_ibtracs()`	25-34	Reads Western Pacific tracks for 2016-2020 near the Philippines (main tracks only) and picks one wind speed per fix (US, else WMO, else Japan).
`gale_radius()`	37-48	Radius of gale-force winds per fix: the largest US 34-kt quadrant, else Japan's 30-kt radius, else a fixed 150 km; fixes below gale strength get no radius. Keeps which source was used. *Why:* the US radius is missing on 15-58% of the fixes of the backtest storms.

BLOCK	LINES	EXPLANATION
`storm_track()`	51-61	Returns the fixes of one backtest storm (by name and season) or any storm id, in time order.
`shift_track()`	64-70	A "what if" track: the same storm moved by `dlat` / `dlon` degrees (and optionally in time). Used for the swath and the map.
`interpolate()`	73-78	Fills in hourly positions between the 3-hourly fixes, so swaths have no gaps.
`swath()`	81-93	The area that saw gale-force winds: the union of circles of the gale radius around every hourly position (in metres).
`track_line()`	96-97	The track as a line (for maps).
`unit_week_exposure()`	100-134	For every unit and week 2016-2020: the closest any storm came, whether the unit was inside a gale radius, the strongest wind within 300 km, and how many storms passed within 300 km. These become forecast features.
`origin_exposure()`	137-153	The forecast's 4-week storm features at a forecast origin (`storm_gale_4w` , `storm_wind_4w` , `storm_dist_4w`), computed exactly as the panel and `forecast.base_features` compute them, from the fixes in the 4 DOH weeks ending at the origin. No fix after the origin week is used.
`shift_storm()`	156-163	The whole IBTrACS table with one backtest storm's fixes moved; every other storm unchanged. Feeding it to `origin_exposure()` scores a shifted track with the same rules as training.

4.5 bantay/rain.py

Daily rainfall grids to weekly rainfall per province.

BLOCK	LINES	EXPLANATION
`CHIRPS_DIR` , `IMERG_DIR`	18-19	Input folders.
`zonal_weights()`	22-40	A sparse matrix saying which grid cells belong to which province (cell centre inside the polygon, land cells only), each row summing to 1. Tiny islands with no cell centre get their nearest land cell. Built once, reused for every day.
`apply_weights()`	43-50	Multiplies a stack of daily grids by the weights: province mean rainfall per day, ignoring missing cells.
`_grid_transform()`	53-58	Builds the pixel-to-coordinate transform from the grid's centre coordinates, and notes whether latitude runs upwards (then the grid is flipped).
`nearest_cells()`	61-68	For a list of points (river reaches), the nearest land cell, via a KD-tree.
`chirps_daily_units()`	71-100	Reads each CHIRPS year file (with the pure-Python `scipy` NetCDF reader, because the netCDF4 library is blocked on some Windows machines), converts its julian-day time axis to dates, and returns daily province rainfall; optionally also the rainfall at each river reach.
`imerg_daily_units()`	103-122	Same for the IMERG monthly GeoTIFF stacks from Earth Engine (one band per day).
`weekly()`	125-139	Daily to DOH weeks: Monday-Sunday total (`_7d`), Friday-Sunday total (`_3d`) and wettest day (`_max1d`).

4.6 bantay/rivers.py

Rain routed downstream, so a province at a river mouth "feels" rain that fell upstream.

BLOCK	LINES	EXPLANATION
`HR_ZIP` , `HR_LAYER`	15-16	The HydroRIVERS Australasia shapefile.
`load_reaches()`	19-35	Reads river reaches in the Philippine box and assigns each to the unit containing its midpoint (reaches just off the coastline go to the nearest unit within 2 km). Sorted by upstream area, which puts every reach before the reaches downstream of it.

BLOCK	LINES	EXPLANATION
`accumulate()`	38-47	Walks reaches from upstream to downstream, adding each reach's rain volume (rain x local catchment area) to the next reach down. Dividing by upstream area gives the average rain over everything upstream of each reach.
`unit_week_river()`	50-69	Weekly version per unit: discharge-weighted average of upstream rain over the unit's reaches (`river_up_7d`) and how much more that is than the unit's own rain (`river_excess_7d`).

4.7 bantay/grids.py

Population grids aligned with WorldPop (100 m), so population, flood and hazard line up cell for cell.

BLOCK	LINES	EXPLANATION
Constants	17-20	WorldPop file, the unit-id raster, the 1 km population cells, and the processing block size.
`fresh()`	23-27	True if a cached file exists and is newer than every input it was built from (e.g. `units.gpkg`), so a changed input triggers a rebuild.
`unit_index()`	30-33	Gives each unit a small integer code (1-85) for rasters; 0 means outside every unit.
`build_unit_raster()`	36-54	Burns the unit polygons into a raster on the exact WorldPop grid, block by block (the full grid is 230 M cells). Reused while it is `fresh()` .
`iter_blocks()`	57-68	Walks several aligned rasters together, one block at a time, to keep memory low.
`worldpop_clean()`	71-74	Turns nodata and negative values into 0 people.
`unit_population_worldpop()`	77-87	Population per unit from WorldPop; the fallback when the census table is missing.
`build_pop_cells()`	90-136	Sums WorldPop into ~1 km cells (369,121 cells, 109 M people), gives each its unit (centre cell, or any unit in the block for coastal cells) and its municipality (containing polygon, else nearest). These are the "people" the access model routes to hospitals. Cached in `pop_cells_1km.parquet` .
`HRSL` , `HRSL_CELLS` , `HRSL_ROWS`	141-144	The two Meta HRSL 2020 layers (children under 5, people aged 60+), their cache, and how many rows of the ~30 m rasters are read at once.
`_hrsl_block_sums()`	147-177	Sums one HRSL raster (read inside its zip, top to bottom) onto the same 1 km blocks `build_pop_cells()` uses: each 30 m pixel goes to the block holding its centre. Also returns the total and the people outside the WorldPop grid.
`build_vulnerable_cells()`	180-212	Under-5 and 60+ people per population cell (same rows as the cells), both layers read in parallel, cached in `pop_cells_hrsl.parquet` . People in blocks without a WorldPop cell are dropped and the kept share is recorded; a layer whose zip is missing stays NaN.
`ADM3_RASTER` , `adm3_index()`	217-222	The municipality raster's path and its stable code per municipality (1..n; 0 = outside).
`build_adm3_raster()`	225-250	Burns the COD-AB municipality polygons onto the WorldPop grid (uint16 codes), block by block, only rasterising the polygons that touch each block; written to `.part.tif` and renamed. Used for Sentinel-1 per municipality.
`_box()`	253-256	A shapely box from raster-window bounds (for the spatial-index query above).

4.8 bantay/flood.py

Flood exposure: UP NOAH hazard maps (static), Copernicus EMS and Sentinel-1 (observed, per typhoon).

BLOCK	LINES	EXPLANATION
`NOAH` , `EMS` , `S1` , `RPS`	28-31	Input folders and return periods (5-yr, 25-yr).
`hazard_raster_path()`	34-35	Where the national hazard raster for a return period is cached.
`_nk()`	40-41	Name normaliser for NOAH file names.

BLOCK	LINES	EXPLANATION
`noah_file_units()`	44-68	Works out which units each NOAH zip covers, from its shapefile names (CamelCase province names or old 9-digit codes) including multi-province files (Mindoro, Metro Manila, pre-split Maguindanao).
`_hazard_col()`	71-72	Finds the hazard-class column (usually `Var`).
`build_hazard_raster()`	75-123	One national raster per return period on the WorldPop grid: **255 = no map, 0 = mapped and safe, 1-3 = low/medium/high hazard**. Starts all-255, then for each province file marks its units as mapped and burns in the hazard polygons (highest class wins). Rebuilt only when a NOAH file or the unit polygons are newer. *Why 255:* "no map" must stay missing, never become "no hazard".
`unit_hazard_shares()`	126-156	Share of each unit's people in any hazard class and in the high class; NaN when less than half the unit's people are on a mapped area (11 units have no 25-yr map).
`ems_observed_flood()`	161-177	Reads the observed-flood polygons from the Copernicus EMSR312 packages (Ompong; first map and later monitoring map).
`s1_tiles()`	182-184	Lists a storm's Sentinel-1 flood tiles.
`_tile_cell()`	187-190	The nominal 1-degree cell of a tile, read from its name (`s1flood_121E_17N.tif`).
`_s1_zone_sums()`	193-218	The shared counter: resamples each tile of a storm onto the WorldPop grid and adds up, per zone code (unit or municipality), the pixels the radar saw, the flooded pixels and the people in them. With `crop`, each tile is read only over its 1-degree cell, so the ~2-pixel overlap between neighbouring tiles is counted once.
`s1_unit_flood()`	221-235	Per unit: share of the land Sentinel-1 saw that newly flooded, and people in that flood (WorldPop). Uncropped, as the panel was built.
`s1_adm3_flood()`	238-276	The same per municipality for every storm with tiles (flood share, people in flood, and `seen_frac`, the share of the municipality the radar saw), on the municipality raster with cropped tiles. Written to `s1_adm3.parquet` by the scenarios stage.
`s1_flood_lookup()`	279-301	Loads a storm's tiles into memory once and returns a function that answers "flooded or not" for millions of points at once with array indexing (road midpoints, friction cells).

4.9 bantay/panel.py

Step 0: the one table every model reads (85 units x 262 weeks).

BLOCK	LINES	EXPLANATION
`PANEL`, `S1_BY_STORM`, `CARD`	15-17	Output files: the panel, the per-storm Sentinel-1 table, and the panel's data card.
`COLUMNS`	19-44	Every panel column with its source and meaning; it drives the data card and guarantees every documented column exists (NaN when its input is missing).
`_newest()`, `cached()`	47-61	A small cache: reuse an intermediate in `data/interim/` if it is newer than all its inputs, else rebuild it. Missing inputs (`None`) are never cached, so they are retried next time.
`rain_layers()`	64-89	CHIRPS weekly rain plus river routing in a single pass over CHIRPS (the slow part), and IMERG weekly rain; both cached.
`static_layers()`	92-98	Builds the unit raster and NOAH hazard rasters (first run only) and returns hazard shares per unit.
`s1_layers()`	101-117	Sentinel-1 flood per unit, placed on 4 weeks starting with the DOH week that contains landfall + `S1_AFTER_DAYS` (the week the map is complete), not the landfall week, so no feature at a forecast origin uses radar taken after it. Writes the per-storm values to `s1_unit_storm.parquet`. Rolly and Ulysses landed 10 days apart, so their windows overlap; the larger flood per unit-week is kept.

BLOCK	LINES	EXPLANATION
`build_panel()`	120-171	Assembles the panel: the full unit x week grid, cases, population (census, else WorldPop), incidence per 100,000, storm exposure (weeks with no storm nearby get "far": 2000 km, no hit), rain, rivers, hazard, the storm-x-hazard interaction and Sentinel-1. Writes the parquet and the data card.
`write_card()`	174-187	Writes `panel_card.md` : coverage of every column and the status of every input.
`load_panel()`	190-191	Reads the panel.

4.10 bantay/leadlag.py

Layer 2: which provinces' dengue rises first, and which follow.

BLOCK	LINES	EXPLANATION
`TAU_MAX` , `MIN_T` , `EDGE_COLS`	16-19	Longest lead considered (3 weeks); the fewest differenced weeks a graph is learned from (52; shorter windows, such as the catalogue's 2017 storms, get an empty graph); the columns of an edge table.
`prepare()`	22-31	For a training window: log incidence, minus the same week a year earlier (removes the shared rainy-season peak, which would otherwise link every pair), drops units with over 10% missing weeks, fills short gaps, standardises each series.
`_bh()`	34-41	Benjamini-Hochberg false-discovery correction of p-values.
`xcorr_graph()`	44-63	The simple baseline: correlation between province i lagged 1-3 weeks and province j now; keeps positive links that survive false-discovery control. Gives 4,265 links on 2016-2019, mostly the shared 2019 epidemic.
`pcmci_graph()`	66-91	PCMCI+ (Tigramite, partial correlations): a link i -> j is kept only if it holds after conditioning on j's own past and other provinces. Keeps lagged links between different provinces with q < 0.05 (on pure noise, PCMCI+ alone returned 60-90 false links). Optional bootstrap gives each link a confidence.
`node_metrics()`	94-110	PageRank on the reversed graph (high score = many provinces follow you: a sentinel), number of leaders and followers, and Louvain communities (provinces that move together). A sentinel is a top-ranked unit (top 10%, at least 5) with at least one follower.
`learn()`	113-123	Runs the above for a window ending at a given date and returns links, per-province metrics and a summary (series, weeks, link count).

4.11 bantay/forecast.py

The forecaster, its baselines, the scores, the rolling-origin test, the calibration of R and the benchmark scoring.

BLOCK	LINES	EXPLANATION
`Q` , `IQ50` , `HORIZONS` , `SURGE_PCTL`	30-33	The 23 quantiles, the index of the median, horizons 1-4 weeks, and the surge definition (above the unit's 90th percentile).
`GROUPS` , `GRAPH_FEATURES` , `ENV_FEATURES` , `MODELS`	35-48	Every feature, grouped into drivers shown on the dashboard (recent cases, season, rain, river, storm/flood, outbreak graph, unit). The built-in models: `xgb` (all features), the ablations `xgb_nograph` (no graph features) and `xgb_noenv` (no rain, river, storm or flood features), and `naive` .
`CAL_MIN_ROWS` , `R_CLIP` , `CUM4_FILE`	52-54	R is recalibrated only once 850 past surge outcomes exist (10 weeks x 85 units); the clip of calibrated R; the file the forecast stage saves the rolling-origin surge forecasts to.
`ENSEMBLE` , `BENCH_GLOB` , `BENCH_MODELS` , `BAND`	55-60	The median ensemble's name, the benchmark file pattern, **the benchmarks allowed into the ensemble** (`chronos2` , `tabpfn_ts` , `mpnn_lstm` ; any other `bench_*` file is a variant and is ignored here), and the fan-chart quantile levels.

BLOCK	LINES	EXPLANATION
`base_features()`	65-102	Features known at the forecast week t (never later): log incidence now and 1-3 weeks ago, 4- and 8-week averages, 4-week change, rain lags and 4/8-week totals, river rain, storm exposure over the last 4 weeks, hazard shares, Sentinel-1 flood, population, and the province as a category. Features whose source is missing are left out.
`target_frame()`	105-116	Adds the target for horizon h (log incidence at t+h), the same week last year (the seasonal-naive value) and its 5-week centred average, and the week-of-year of the target.
`cum4_frame()`	119-127	Target = total incidence over the next 4 weeks (used for the surge probability R).
`graph_features()`	130-150	For each follower province, its leaders' recent rise and year-on-year anomaly (weighted by link strength), and the number of leaders. Uses only leader values up to week t.
`feature_columns()`	153-159	The feature list, with or without the graph features or the environmental features (the ablations).
`fit_quantile_model()`	164-169	One XGBoost model that outputs all 23 quantiles at once (quantile loss), 150 trees, depth 4.
`predict_quantiles()`	172-173	Predicts and sorts each row's quantiles (so they never cross).
`usable()`	176-178	Drops features that are empty in the training rows.
`seasonal_naive()`	181-185	The bar to beat: "same week last year", with a range taken from its past errors.
`prob_exceed()`	188-200	Turns 23 quantiles into the probability of exceeding a threshold, by interpolating the quantile curve.
`surge_thresholds()`	203-208	Each unit's 90th percentile of 4-week incidence, computed from training weeks only.
`to_cases()`	213-214	Back from log incidence per 100,000 to case counts.
`wis()`	217-227	The weighted interval score (Bracher et al., 2021): rewards narrow ranges that contain the truth and penalises misses in proportion to how far out they are.
`coverage()`	230-233	Whether the outcome fell inside the 50% or 90% range.
`refit_dates()`	238-239	Refit every 13 weeks from January 2018 to December 2019 (8 refits).
`rolling_origin()`	242-282	**The honest test.** At each refit date: re-learn the graph and thresholds on past data, train on rows whose target is before the refit date, forecast every week until the next refit. Per horizon it fits `xgb`, `xgb_nograph` and `xgb_noenv` (the surge model only the first two) and adds the seasonal naive; when a window's graph has no links, `xgb` equals `xgb_nograph`, so that fit is skipped. The 4-week model gives the raw surge probabilities `R_xgb`, `R_xgb_nograph`.
`score_rows()`	285-297	WIS, absolute error and 50%/90% coverage on case counts for any set of models' quantiles; a row with a missing quantile scores NaN, so models can be compared on the same rows.
`score()`	300-321	Scores every forecast of the built-in models, plus an online conformal 90% band (built only from errors of forecasts already past) as a check.
`summarise()`	324-345	Averages by horizon and overall, every model on the same rows: WIS, relative WIS vs seasonal naive, graph vs no-graph, with vs without environment, coverage.
`SurgeCalibrator`	348-361	An isotonic map from raw R to the observed surge frequency, fitted on past forecasts only; keeps how many outcomes it used and the last target week it saw.
`SurgeCalibrator.__init__()`	352-354	Fits the isotonic regression (increasing, clipped to 0-1).
`SurgeCalibrator.__call__()`	356-361	Maps raw R to calibrated R (NaN stays NaN, result clipped to `R_CLIP`).
`fit_calibrator()`	364-372	Builds a calibrator from rows whose outcome is known at a given date (target week on or before it, the same rule as model training); None with fewer than `CAL_MIN_ROWS` outcomes or only one class.

BLOCK	LINES	EXPLANATION
`cum4_eval()`	375-400	The evaluation version: for every forecast origin t, calibrates that origin's R with a calibrator fitted on forecasts whose 4-week window ended by t (expanding window), so calibrated R never uses a later outcome; rows before 850 past outcomes keep the raw R. Asserts there is no leakage.
`RELIABILITY_BINS`, `_reliability()`	403-408	Reliability table: in bins of R, the mean R and how often a surge happened.
`_brier()`	411-412	Brier score of a probability column.
`surge_calibration()`	415-443	Brier score of raw and calibrated R, the base rate, the climatology Brier, which origins stayed uncalibrated, and the reliability table before and after calibration.
`surge_calibrator()`	446-461	The production version for the scenarios: a calibrator fitted on every saved rolling-origin surge forecast (`eval_cum4.parquet`) whose outcome is known by the model's training end. None (R stays raw, recorded) if the file is missing or there are too few outcomes.
`calibrate_R()`	464-472	Scenario hook: keeps the model probability as `R_raw`, sets `R` to the calibrated value (raw if there is no calibrator) and flags `R_calibrated`.
`eval_quantiles()`	477-484	The rolling-origin horizon 1-4 rows with every built-in model's 23 quantiles, saved so benchmarks can be scored and ensembled later without refitting XGBoost.
`_key()`	487-490	The join key (unit, origin week, horizon) in one fixed set of types.
`load_bench()`	493-539	Reads each `bench_<model>.parquet` in `BENCH_MODELS` (and its `.json` metadata), checks the schema, drops invalid or duplicate rows and aligns it to the evaluation rows. Other names are recorded as ignored variants; a bad file is skipped with a warning, never fatal.
`evaluate_models()`	542-583	Scores xgb, its ablations, the naive baseline, every benchmark and the **median ensemble** (element-wise median of the 23 quantiles over xgb and the benchmarks, only where all exist) on the same rows. Returns the model x horizon table, fan-chart bands for the benchmarks and the ensemble, and metadata.
`fit_at()`	588-603	For the scenarios: learns the graph and the surge thresholds and trains the graph model for every horizon and the 4-week model on data up to a storm's forecast origin (never past 2019).
`predict_at()`	606-631	Forecasts every province from that week; optional overrides replace the storm features (one row per unit); computes raw R and the per-driver SHAP contributions shown on the dashboard.

4.12 bantay/access.py

Layer 3: travel time to the nearest hospital over roads and ferries, before and during a storm, by two methods: a road and ferry **network** (the primary method) and a **raster** over the MAP friction surface (the check). The gap between them is reported as uncertainty.

BLOCK	LINES	EXPLANATION
`PBF`, `ROADS`, `SPEED`	35-40	The OpenStreetMap file, the cached road graph, and assumed travel speeds per road class (links at 80% of their road). Speeds are a tunable assumption, tested in the sensitivity analysis.
`WALK_KMH`, `MAX_SNAP_KM`, `LAT0`	42-43	Walking 5 km/h from a home to the road, at most 5 km; a reference latitude for fast km conversions.
`_xy()`, `_hav_km()`	46-54	Coordinates to km (for nearest-node search) and great-circle distance.
`build_road_graph()`	59-96	Reads OSM roads through GDAL, finds junctions as coordinates shared by two or more roads (OSM roads share nodes there), cuts roads into segments between junctions, and stores each segment's length, speed and midpoint. 1.72 M junctions, 2.13 M segments, cached.

BLOCK	LINES	EXPLANATION
`_nhfr()`	101-122	Reads the DOH facility registry if downloaded by hand: finds latitude/longitude and type columns and keeps hospitals.
`load_hospitals()`	125-155	NHFR if available, else the fallback: OSM hospitals plus NOAH facilities whose names say hospital or medical center, with duplicates within 300 m removed (2,199). Labelled as a fallback everywhere.
`load_ferries()`	160-169	The 108 RoRo routes; crossing time plus half the time between departures (capped at 12 h), and a straight line between ports for storm checks.
`Network`	174-225	The routable road and ferry network.
`Network.__init__()`	177-200	Snaps hospitals and population cells to the nearest road junction (within 5 km; a cell further from a road has no route) and ferry ports to junctions within 25 km (their coordinates are coarse), adding the port access at 30 km/h to the crossing time.
`Network.road_hours()`	202-207	Travel hours per segment, with optional speed multipliers (0 = road closed).
`Network.solve()`	209-225	Builds a sparse graph of roads and ferries (keeping the fastest of parallel edges) and runs one multi-source Dijkstra from all hospitals at once: hours from every junction to its nearest hospital. Adds the walk from each population cell.
`cut_off()`	228-230	A cell is cut off if it had a route before the storm and now has none, or its trip at least doubled and grew by more than an hour.
`ferry_keep_mask()`	235-244	Which ferry routes still sail: cancelled if the route crosses the gale swath (or, in the sensitivity variant, if a port is inside it).
`edges_in()`	247-254	Which road segments lie inside the swath (vectorised point-in-polygon on their midpoints; the raster method reuses it for cell centres).
`hazard_at()`	257-276	NOAH hazard class at each segment midpoint, read in bands to save memory.
`road_multiplier()`	279-289	Speed multiplier per segment: observed flood closes roads; otherwise (shifted tracks, catalogue storms) high hazard inside the swath closes and medium hazard slows. Returns which rule was used.
`FRICTION` , `SEA_MIN_PER_M` , `RASTER_SNAP_KM`	294-299	The MAP motorised friction GeoTIFF; the constant MAP uses for open sea (3.16 min/km, not nodata), which is treated as impassable outside the unit polygons; how far a population cell on a sea cell may snap to land (3 km).
`FrictionGrid`	302-399	**The raster access method**: cost-distance from the same hospitals over the ~1 km friction grid, with the same ferries, storm rules and cut-off rule as the network, evaluated at cell centres. There is no walking leg.
`FrictionGrid.__init__()`	316-371	Turns friction (minutes per metre) into hours per cell, marks sea impassable, draws each RoRo route as a line of sea cells whose cost adds up to the network's crossing time, and places hospitals and population cells on the grid (a cell on sea snaps to the nearest land cell within 3 km, else it has no route).
`FrictionGrid.hazard()`	373-377	NOAH hazard class at the centre of every passable land cell (computed once).
`FrictionGrid.in_swath()`	379-380	Which land cells lie inside the swath (reuses `edges_in()`).
`FrictionGrid.solve()`	382-399	Applies the storm multipliers (flooded cells impassable, slowed cells costlier) and the ferries that still sail, then runs scikit-image's `MCP_Geometric` from all hospital cells: hours to the nearest hospital per population cell. A hospital in a flooded cell is not a start.
`friction_grid()`	402-406	Builds the `FrictionGrid` , or records the friction raster as missing and returns None.
`summarise()`	411-423	Per unit (or municipality): population, people cut off, people with no route even normally, average normal trip time, and A = share cut off.

4.13 bantay/scenarios.py

Puts it together: for each typhoon and track, R x A = P, action tiers, then the checks; and the same for the storm catalogue.

BLOCK	LINES	EXPLANATION
`SHIFTS`	30	The 15 tracks per storm: north/south 0, ±0.5, ±1 degree x east/west 0, ±1.
`TIERS` , `TIER_LABELS`	31-34	Provisional action rules: deploy if R >= 0.5 and A >= 10%; pre-position if R >= 0.3, or A >= 10% and R >= 0.15; else monitor.
`tier()`	37-42	Applies the rules to every province.
`overrides_for()`	45-52	The origin-week storm features (gale hit, wind, distance over 4 weeks, storm x hazard) for this storm moved by (dlat, dlon), built like the training features (`storms.origin_exposure()`). Used for every track, the unshifted one too, so tracks differ only by the shift. Observed rain is kept.
`AccessModel`	55-142	Builds the network (and the raster method, when the friction GeoTIFF is on disk) once and reuses it for every scenario.
`AccessModel.__init__()`	59-78	Road graph, hospitals, ferries, the normal-day travel times of both methods, hazard per segment and EMS polygons. If an input is missing, A stays NaN.
`AccessModel.observed_flood()`	80-91	Observed flood for a historical storm at road midpoints or at friction-cell centres, cached per storm (sensitivity asks many times).
`AccessModel._observed_flood()`	93-103	Sentinel-1 if its tiles are on disk, else the EMS areas for Ompong, else nothing.
`AccessModel._multiplier()`	105-116	The flood rule, the same for both methods: shifted tracks and storms without observed flood use the NOAH proxy; an unshifted storm with Sentinel-1 uses only the observed flood; EMS (a few areas only) is added on top of the proxy. Returns the flood source label.
`AccessModel.run()`	118-129	One storm scenario on the network: cancel ferries in the swath, cut or slow roads by the flood rule, re-solve, mark who is cut off. Options are the sensitivity knobs.
`AccessModel.run_raster()`	131-142	The same scenario on the raster (same swath, same cancelled ferries, same flood rule); None without the raster.
`VULNERABLE`	145	The Meta HRSL columns (under 5, 60+) carried through when they are on the population cells.
`access_tables()`	148-170	Access summary per unit or municipality: the network numbers (A, people cut off, trip time), the raster ones (`*_raster`), `A_lo` / `A_hi` = the lower and higher of the two (NaN without both), and under-5 / 60+ people and those cut off.
`track_access()`	173-187	Both methods for one track: unit and municipality tables, flood source, ferries cancelled, and a row comparing the methods (mean A, people cut off, rank agreement) for `access_methods.csv` .
`baseline_agreement()`	190-212	Normal-day comparison of the two methods: share of people who can reach a hospital, mean trip time, correlations across cells and provinces, typical raster-to-network ratio.
`_wmedian()`	215-218	Weighted median (for the population-weighted ratio).
`case_columns()`	221-226	Expected cases over the next 4 weeks (median and 90% range) from the 4-week quantiles.
`scenario_rows()`	229-238	One track's province rows: forecast + access + P = R x A, `P_lo` / `P_hi` (the method range), tier and sentinel status.
`adm3_priority()`	241-244	Municipal rows get their province's R; P = R x municipal A (and its range).
`write_scenario_outputs()`	247-260	Writes `scenarios{suffix}.parquet` and its companions (municipalities, tracks, swaths, ferry status, access methods); the suffix is empty for the backtests and `_catalog` for the catalogue.

BLOCK	LINES	EXPLANATION
`run_all()`	263-309	For each backtest storm: forecast origin = the DOH week containing landfall, models trained to that week (2020 storms share the 2019 fit), R calibrated on past forecasts (`R_raw` kept); for each of 15 tracks: R and expected cases, A per province and municipality by both methods, P, tier, sentinel status; saves everything the dashboard needs, plus the ferry routes and hospitals.
`CATALOG_MIN_WIND_KT` , `CATALOG_MIN_HISTORY_WEEKS` , `CATALOG_REFIT_WEEKS`	314-316	A catalogue storm needs a gale-strength fix (34 kt) and a model with at least 52 weeks of history; fits are cached on a 13-week grid.
`catalog_train_ends()`	319-325	The training ends the catalogue may use: the forecast's refit dates, extended back in 13-week steps to the panel start, plus `MODEL_END` .
`catalog_storms()`	328-351	Every Western Pacific storm 2016-2020 that came within 1,500 km: name, season, max wind, closest approach and its DOH week (the forecast origin), and whether its whole-track gale swath meets land.
`run_catalog()`	354-426	The unshifted scenario for every catalogue storm except the 4 backtests. The model is the latest catalogue fit on or before the origin (never past 2019), so no training target is after the origin; storms with under 52 weeks of history are skipped and listed in `catalog_storms.csv` . Flood on roads and cells is the NOAH proxy (no Sentinel-1 map for these storms).
`validate()`	429-450	Backtest: did provinces with high R / P see more cases in the next 4 weeks? Spearman correlations, surge rate of the top 10, Brier score. 2020 storms are flagged.
`_rank_change()`	453-458	Kendall's tau and top-10 overlap of a changed ranking against the base one.
`sensitivity()`	461-491	Re-runs access under 9 changed assumptions (speeds, ferry rule, slow-down, cut-off rule, storm radius) on both methods and measures how much the ranking moves.
`metro_manila_benchmark()`	494-508	Closes every road in NOAH medium/high flood hazard and compares the share of Metro Manila cut off (about 30% in this build) with the published ~22% (Tariverdi et al., 2023): an order-of-magnitude check.

4.14 bantay/lepto.py

The leptospirosis stretch goal: does leptospirosis follow rain and typhoons, and after how many weeks? A retrospective association analysis, not a forecast: every model is fit on the whole overlap with the rain data. Two series: national weekly counts (Mendeley, 2019-2020 overlap with CHIRPS) and city-week counts for the 12 CCHAIN cities (2016-2020), which are never read as provincial or national numbers.

BLOCK	LINES	EXPLANATION
Paths	43-47	The Mendeley workbook, the CCHAIN folder, the DOH dengue file (for date checks), the summary output and the city-rain cache.
Analysis constants	49-59	Lags 0-6 weeks; one IRR unit = +50 mm/week of rain; heavy-rain week = at or above the 90th percentile; 2 seasonal harmonics; 4-lag robust errors; COVID term from 2020-03-15; CCHAIN week label = date + 13 days; event window -4..+8 weeks and its four periods; 2014-2019 as the seasonal baseline years; 0.1 cases/week floor for a zero expectation.
`HDX_CITY`	61-64	The DOH dengue name of 10 of the 12 CCHAIN cities (for the date check).
`first_sunday()`	69-71	First Sunday of a year.
`mendeley_weekly()`	74-85	National weekly counts of one disease from the Mendeley workbook; morbidity week w of year Y is the DOH week ending first Sunday + 7(w-1) days.
`cchain_weekly()`	88-104	CCHAIN city-week counts of one disease, with the old 9-digit code turned into the COD-AB code and the week labelled as the CCHAIN date + `label_days` .
`_new_pcode()`	107-109	Old 9-digit PSGC to COD-AB municipality code.
`hdx_dengue()`	112-118	The DOH dengue file, read with the usual rules, for the date checks.

BLOCK	LINES	EXPLANATION
`alignment()`	121-143	Which week labels are right? Correlates the same files' dengue counts with the DOH file under several shifts: Mendeley vs the DOH national total, CCHAIN vs the DOH city rows. This is where `CCHAIN_LABEL_DAYS` = 13 comes from.
`_r()`	146-151	Correlation of two weekly series, on levels and on week-to-week differences (the sharper timing test).
`city_polygons()`	154-168	COD-AB polygons of the 12 CCHAIN cities.
`city_rain()`	171-184	CHIRPS weekly rain per city (the `rain.py` functions on the city polygons), cached in `data/interim/`.
`city_storms()`	187-194	Weekly gale exposure of each city from every storm (`storms.unit_week_exposure()` on the city polygons).
`national_exposure()`	197-215	National weekly rain (area-weighted CHIRPS and IMERG over the 85 units, plus population-weighted rain) and the population share inside a gale swath.
`harmonics()`	220-222	Sine and cosine terms of the day of year (the seasonal adjustment).
`add_lags()`	225-233	Lagged copies of a column, checking that the shifted row is exactly k weeks earlier.
`fit_nb()`	236-250	Negative binomial (NB2) fit with Poisson start values, with Newey-West (national) or Driscoll-Kraay (cities) standard errors.
`_irr_rows()`, `_cum_row()`	253-274	Incidence rate ratios with 95% CI per term, and the cumulative effect of lags 0-6 (sum of the joint coefficients, with its own CI).
`lag_profile()`	277-295	For one exposure: lags 0-6 fitted one at a time ("single") and together ("joint", a distributed-lag model) plus their sum, all on the same weeks.
`national()`	300-349	The national analysis for 2019-2020: rain, population-weighted rain, heavy-rain weeks and gale share, with sensitivity fits (2019 only, weeks that look completely reported, no seasonality).
`city_panel()`	354-375	The complete city x week table: cases, rain, heavy-rain flag (each city's own 90th percentile), gale hit and their lags.
`city_lags()`	378-392	The city design: city and year fixed effects, COVID term, common (or per-city) seasonal harmonics.
`cities_analysis()`	395-423	Lag profiles for rain, heavy-rain weeks and gale hits in the 12 cities, plus sensitivity fits: without Iloilo City (about half of all cases), city-specific seasons, and the literal CCHAIN week label (date + 6).
`s1_city()`	428-455	Sentinel-1 flooded share of each city for one storm (descriptive only).
`_expected()`	458-471	Expected cases for a city-week: the city's mean in the same 3 calendar weeks of the other 2014-2019 years.
`events()`	474-506	The event-study table: for each backtest storm, each city and weeks -4..+8 around landfall, whether the city was inside the storm's gale swath, observed and expected cases, rain and Sentinel-1 flood.
`event_did()`	509-542	Quasi-Poisson model with offset log(expected) and storm x week effects: the IRR of inside vs outside cities per period. The before-landfall period is a placebo and should be about 1.
`event_summary()`	545-564	Per storm and period: observed and expected cases inside and outside, and their ratio with a simple Poisson CI.
`_fmt()`	569-570	"IRR (lo-hi)" text.
`run()`	573-615	The whole stage: writes the `lepto_*` tables and series to `data/processed/app/` and the summary.

BLOCK	LINES	EXPLANATION
`_lag_md()`, `_peak()`, `_sig_lags()`, `_span()`, `_findings()`	618-704	Markdown lag tables, the peak lag of a joint profile, the lags whose 95% CI lies above 1, and the key-findings bullets for the summary. Every qualitative word in the findings (the lag window, "same direction", a placebo that "passes" or "fails", "no general storm effect") is decided from the numbers, so the summary stays right if the data change.
`write_summary()`	707-825	Writes `data/processed/lepto_summary.md`: key findings, data and week labels, the national and city tables, the event study and the caveats; every number from the output tables.

5. THE BUILD: scripts/build.py

One command runs the stages in order; each can also be run alone.

BLOCK	LINES	EXPLANATION
Docstring	1-23	What each stage writes; the same text is the `--help`.
Imports and warning filters	25-43	Adds the repo root to the import path (so `bantay` imports from any folder) and silences known harmless warnings.
`STAGES`	44	The stage order: crosswalk, panel, leadlag, forecast, ensemble, scenarios, catalog, app, lepto.
`stage_crosswalk()`	47-52	Crosswalks (reused if present), the optional PSGC check, unit polygons. Stops the build if the crosswalks can't be written (COD-AB or the dengue file missing), because everything depends on them.
`stage_panel()`	55-57	Builds the panel.
`stage_leadlag()`	60-69	The final graph on all 2016-2019 data (PCMCI+ with `--boot` bootstrap samples, 20 by default, and the correlation baseline), saved for the dashboard and the scenarios.
`stage_forecast()`	72-106	Runs the rolling-origin test and saves the scores, the summary, the calibration and reliability of R, the rolling-origin surge forecasts (for the scenarios' calibration), every model's full quantiles (for the ensemble stage), the graph of each refit and compact forecasts (5/25/50/75/95% in cases) for the fan charts. Then runs `stage_ensemble()` unless that stage was also asked for.
`stage_ensemble()`	109-127	Re-scores without refitting: `forecast.evaluate_models()` on the saved quantiles plus any benchmark files; writes the model table, its metadata and the benchmark fan-chart bands (deleting stale bands when no benchmark is left).
`population_cells()`	130-137	The 1 km population cells with the HRSL under-5 / 60+ columns attached (NaN if HRSL is missing).
`stage_scenarios()`	140-171	Population cells, hazard shares, all scenarios, the method comparison printout, Sentinel-1 per municipality, validation, sensitivity and the Metro Manila benchmark.
`stage_catalog()`	174-180	The storm catalogue (`scenarios.run_catalog()`).
`stage_app()`	183-187	A lighter province map for the dashboard.
`stage_lepto()`	190-199	The leptospirosis analysis (`lepto.run()`); prints the national and city rain lag profiles.
main block	202-221	Parses stage names and options (`--force`, `--boot`, `--refit-weeks`), rejects unknown stages, runs the chosen stages in `STAGES` order with timings, then lists every missing input.

6. THE DASHBOARD: app/streamlit_app.py

Streamlit re-runs the whole script top to bottom every time the user changes a control; `@st.cache_data` keeps the loaded files in memory, so each re-run is fast. The cache is keyed on the files' modification times, so a rebuild shows up on the next page refresh without a restart. The main controls (storm, track, map colour, open tab) are bound to the URL, so a view can be shared as a link. Tabs are a registry at the bottom of the file: each is a function, and optional ones appear only when their files exist.

BLOCK	LINES	EXPLANATION
Docstring	1-12	Layout, the optional outputs the app looks for, and the two environment overrides the tests use (`BANTAY_APP_DATA` , `BANTAY_SUS_PATH`).
Imports, page setup	14-41	Adds the repo root and `app/` to the path, imports the tier rules from `bantay.scenarios` and the helper modules, sets a wide page and the CSS.
`APP` , `STATUS_FILE` , `DRIVERS` , `BASE`	43-48	Where the build outputs are (overridable for tests), the status file, the driver groups whose SHAP values explain each risk, and the columns every scenario row is given (NaN if an older build lacks them).
`stamp()`	52-54	Names and modification times of every output: passed to `load()` so its cache refreshes when any file changes.
`_read()`	57-64	Reads a pre-computed file if it exists (parquet, CSV or JSON), else returns None, so a missing output hides its panel instead of crashing.
`load()`	67-90	Loads every output once (cached): the core files, then the optional ones (access methods, model comparison, Sentinel-1 per municipality, the catalogue, leptospirosis, benchmark summary). Catalogue tracks, swaths, ferries and municipalities are appended to the backtest ones.
`swath_xy()`	93-104	Turns a stored swath polygon into map coordinates.
Load and guard	107-112	If no scenarios exist yet, tells the user to run the build and stops.
Catalogue merge	114-128	Adds the catalogue storms to the scenario table (a wrong file becomes a note on *Data & caveats*, not an error) and keeps each one's name, season and closest approach.
`names` , `PAL`	130-131	Unit id to province name, from the map file; the light or dark palette.
`storm_label()`	134-143	"Ompong (Mangkhut, 2018)" for a backtest storm, "Name Year" (or "Unnamed") for a catalogue storm.
Sidebar	147-190	Typhoon picker (the four backtests first, then the catalogue by date; type to search), track shift sliders (only for storms with shifted tracks) with a reset button, and a "How to read this" box.
Scenario selection	192-208	Picks the 85 rows of the chosen storm and shift, fills missing columns, sorts by P (else R), and works out the flood source (radar = lower bound, hazard proxy = upper bound).
`flag()`	211-212	A missing flag counts as false.
`headline()`	216-247	Title, forecast week, and badges: flood source with its bound, hospital source, training end, 2020 warning, catalogue storm, calibration status.
`caveats()`	250-295	Every caveat that applies to this scenario, most decision-relevant first: 2020 reporting, missing access, radar lower bound or hazard upper bound, shifted track, ferry proxy, hospital fallback, calibration of R, the two access methods, provisional tiers.
Caveat display	298-308	Warnings (or the first note) shown on top; the rest in an expander.
KPIs	311-334	Provinces to deploy to and to pre-position for, people cut off (with its bound), ferries cancelled, top priority; a caption with the raster method's count and the under-5 / 60+ people cut off.
`act_table()` , `ACT_COLS`	338-349	The top-10 "act first" table and the column formats and help texts shared by the tables.
`tab_map()`	352-376	*Action map*: the map (coloured by P, R or A) next to the top-10 table and the tier counts.

BLOCK	LINES	EXPLANATION
`map_figure()`	379-436	The province map in plotly (no map tiles, works offline): grey base layer, the chosen layer, the gale swath, cancelled ferry routes and the track (hover: time and wind); hover on a province shows R, A (with the method range), P and people cut off.
`tab_quadrant()`	439-496	*Risk vs access*: R against A, bubble size = population, colour + legend = tier, whiskers = the two access methods, the threshold lines and the deploy zone, top six named. Zooms in when A is small.
`drivers()`	499-503	The two driver groups that pushed a province's R up most (SHAP above 0.02).
`ranked()`	506-531	The province table: action, P, R (and raw R), A with the method range and raster value, cases next 4 weeks with 90% range, people cut off (both methods, under 5, 60+), sentinel rank, community, main drivers.
`tab_table()`	534-556	*Provinces*: that table with column help, and a CSV download.
`tab_muni()`	559-629	*Municipalities*: for a chosen province (starting with the top one), where to send a clinic first, a bar chart of people cut off and a table per municipality; for the observed track it adds the radar flood share, people in radar flood and radar coverage.
`tab_graph()`	632-671	*Outbreak graph*: the PCMCI+ links with confidence, sentinel ranking, communities, the fit details and the correlation-baseline count for contrast.
`MODEL_NAMES`, `MODEL_ORDER`, `KIND_NAMES`	674-680	Readable model names and their order in the model picker.
`median_wis()`	683-694	Median WIS per model from `bench_summary.csv` (NaN where the build has none).
`tab_skill()`	697-880	*Forecast skill*: the evaluation table (and all scores with the ablations), the model comparison on the same rows (mean and median WIS, benchmark variants in an expander), the Brier score and reliability chart of R before and after calibration, and a fan chart for any province, model and horizon.
`tab_lepto()`	883-884	*Leptospirosis*: hands over to `lepto_tab.render()`.
`access_methods()`	887-958	The two access methods compared: normal-day reach and trip time, their agreement, and people cut off by each for every track (backtests and catalogue), with the current scenario circled.
`tab_validation()`	961-1011	*Validation*: backtest table, sensitivity (Kendall's tau per variant, storm and method), the method comparison, the Metro Manila benchmark.
`tab_data()`	1014-1045	*Data & caveats*: every caveat, problems with optional files, every input with available / missing from `status.json`, and the storm catalogue with the skipped storms and why.
`answer_key()`	1048-1071	The facilitator's answers to the usability task, computed from this build.
`sus_form()`	1074-1093	The System Usability Scale form (office, task outcome, minutes, ten statements); refuses incomplete answers and saves through `sus.save()`.
`tab_sus()`	1096-1130	*Usability test*: the 15-minute scenario task, the answer key, the form and the responses so far (count, mean score, task completion, by office). No names are collected.
`TABS` and the tab loop	1133-1148	The tab registry `(label, function, show?)`; adding a section = adding a line. The Leptospirosis tab shows only when its files exist.

6.1 app/ui.py

Look and formatting for the dashboard, kept apart from the logic.

BLOCK	LINES	EXPLANATION
`LIGHT`, `DARK`	13-26	The data colours for each theme: a single-hue blue scale for magnitudes, blue steps for the three tiers (always shown with a text label), orange for the track and swath, aqua for cancelled ferries. Checked for colour-blind separation.

BLOCK	LINES	EXPLANATION
`TIER_SHORT` , `TIER_TINY`	28-29	Tier names for the KPIs and the tables.
`FLOOD_LABEL` , `HOSPITAL_LABEL`	31-35	Readable names for the flood source used on roads and the hospital source.
`HELP`	38-62	Plain-language help for every term an officer meets on screen (R, A, P, tiers, ranges, sentinels, WIS, coverage, the method range).
`_THEME_JS` , `_watcher`	66-92	A tiny browser component that notices when the viewer switches light/dark and re-runs the script, because Streamlit restyles the page but not the chart colours.
`palette()`	95-102	The light or dark colours, following the viewer's theme (light when unknown, e.g. in tests).
`watch_theme()`	105-117	Mounts the theme watcher once per run and remembers the theme it reports.
`css()`	120-135	A few CSS rules on stable selectors only, so a Streamlit update can't break the layout.
`nan()`	138-139	True for None or NaN.
`people()`	142-151	Rounded people counts for headline numbers (342,801 -> 343K; 7,473,210 -> 7.47M).
`rng()`	154-155	A "lo-hi" range, or n/a.
`date()`	158-162	Dates written day-month-year (16 Sep 2018), as in Philippine health bulletins.
`shift_label()`	165-169	"1° south", "no shift".

6.2 app/sus.py

The System Usability Scale (Brooke, 1996) for the dashboard's user test: items, scoring and anonymous storage.

BLOCK	LINES	EXPLANATION
`ROOT` , `DEFAULT_PATH`	12-13	Responses go to `data/processed/sus/sus_responses.csv` (git-ignored).
`ITEMS` … `FIELDS`	16-31	The ten standard statements (odd ones positive, even ones negative), the 1-5 scale, the offices a tester can pick, the task outcomes and the CSV columns. No name or contact field exists.
`path()`	34-36	The responses file, or `BANTAY_SUS_PATH` if set (the tests write to a temporary file).
`score()`	39-44	The SUS score 0-100: odd items count x - 1, even items 5 - x, the sum times 2.5; rejects anything but ten answers of 1-5.
`save()`	47-65	Appends one response (office, answers, score, task outcome, minutes, UTC time) and returns its score.
`load()`	68-74	All responses so far (an empty table if none).

6.3 app/lepto_tab.py

The *Leptospirosis* tab. It reads only the build's `lepto_*` files; every number on screen comes from them.

BLOCK	LINES	EXPLANATION
Constants	13-20	The caveat shown on top (12 cities, not provincial; association, not forecast), the exposure toggle, the main variant of each analysis, the event periods and the storm pools.
`_profile()`	23-25	The joint lag profile and the cumulative row of one variant and exposure.
`_ci()`	28-29	"IRR (lo-hi)" text.
`findings()`	32-56	One plain sentence per analysis (peak lag in the cities and nationally, the event study), read from the files.
`render()`	59-65	The tab: caveat, findings, then the three charts.

BLOCK	LINES	EXPLANATION
`lag_profile()`	68-103	IRR by lag for cities and national, for rain, heavy-rain weeks or gale exposure, on a log scale with 95% CIs.
`weekly_chart()`	106-141	Weekly cases and rain for the nation or any city, with the four landfalls marked and weeks that look incompletely reported shown lighter.
`event_study()`	144-193	The inside/outside ratio of observed to expected cases by period for each storm pool (the placebo period hollow), and the per-storm ratios in an expander.

6.4 .streamlit/config.toml

The dashboard's settings and theme, read by Streamlit when it starts from the repo folder (`run_app.bat` does that): viewer-only toolbar (no developer or deploy menu for health officers), short error messages without tracebacks, no usage statistics, and the light and dark themes (Public Sans, one flood-blue accent). The data colours live in `app/ui.py`.

6.5 TESTS: tests/

The tests read the real build outputs in `data/processed/app/` (they skip when those are missing) and never write there; usability answers go to a temporary file. Screenshots and videos land in `tests/artifacts/` (git-ignored). Install `requirements-dev.txt` and `python -m playwright install chromium` once, then:

```
python -m pytest tests                  # everything: AppTest + Playwright end-to-end
python -m pytest tests -m "not e2e"     # fast AppTest checks only
python tests/capture.py                 # refresh docs/results/figures/dashboard.png + record the demo walkthrough video
```

`tests/conftest.py`

BLOCK	LINES	EXPLANATION
Paths	17-21	Repo root, the app, the build outputs, the artifacts folder; puts `app/` on the path so the tests can import `sus`.
`needs_data`	23-24	Skip marker for tests that need a finished build.
`pytest_configure()`	27-28	Registers the `e2e` marker, so `-m "not e2e"` deselects the browser tests.
`free_port()`	31-34	A free local port for the test server.
`sus_csv()`	37-41	A throw-away responses file, set through `BANTAY_SUS_PATH`.
`server()`	44-73	Starts `streamlit run` on a free port for the whole session, waits for its health check and stops it at the end.
`shots()`	76-80	The screenshot folder.

`tests/test_sus.py`

Unit tests of `app/sus.py`; they need no build.

TEST	LINES	WHAT IT CHECKS
`test_score_examples()`	8-12	Known SUS scores (all neutral = 50, best = 100, worst = 0, a mixed case = 75).
`test_score_rejects_bad_answers()`	15-18	Nine or eleven answers, or values outside 1-5, raise an error.
`test_ten_standard_items_and_no_identifiers()`	21-24	Ten items, the four offices, and no name, email, phone or organisation column.
`test_save_appends_to_env_path()`	27-36	Responses are appended to the file in `BANTAY_SUS_PATH` with the right columns and scores.

TEST	LINES	WHAT IT CHECKS
`test_save_rejects_unknown_role()`	39-42	A free-text "office" (e.g. a person's name) is refused and nothing is written.

`tests/test_app.py`

Runs the dashboard headless with Streamlit's AppTest (fast, no browser).

BLOCK	LINES	EXPLANATION
Setup	12-16	Every test here needs a build; the expected tabs; the optional tab; which output files exist.
`run()` , `metrics()` , `at()` , `tab()` , `specs()`	19-41	Helpers: run the app and fail on any exception, read the KPIs, the default fixture (real outputs, temporary SUS file), find a tab, get the plotly figures in a block.
`test_every_tab_renders()`	44-56	The tabs are in order, the title is Typhoon Ompong, the five KPIs, charts and tables are there, and no error is shown.
`test_numbers_match_outputs()`	59-65	The KPI counts equal the tiers and ferries in `scenarios.parquet` .
`test_shift_changes_outputs()`	68-77	Moving the track 1° south changes the KPIs and the title and brings the hazard-proxy note; the reset button restores them.
`test_storm_changes_outputs()`	80-86	Picking Rolly changes the numbers and shows the 2020 COVID warning.
`test_map_colour_and_drilldowns()`	89-93	The map colour, municipality picker and fan-chart horizon all work.
`test_sus_form_saves_to_temp_file()`	96-105	A full form saves one row scoring 75 to the temporary file.
`test_sus_form_rejects_incomplete()`	108-112	An incomplete form is not saved.
`needs`	116	Skip marker for tests that need a given optional output.
`test_catalogue_storm()`	119-136	The storm list starts with the four backtests then the catalogue by date; a catalogue storm renders with its track, no sliders, the calibration warning when it applies and the right ferry count.
`test_skipped_storms_listed()`	139-142	The skipped-storms table is shown.
`test_access_methods_shown()`	145-160	With the raster method in the build: the method-range, raster, under-5 / 60+ and raw-R columns, the KPI caption, the municipal radar columns and the method comparison on *Validation*.
`test_calibrated_reliability()`	163-170	The reliability chart shows raw and calibrated R and the calibrated Brier text.
`test_model_comparison_and_benchmark_fans()`	173-189	The model picker lists XGBoost first and every benchmark with a fan chart, each renders, the best model is named, and the note that scenarios use XGBoost is shown.
`test_lepto_charts()`	192-206	The *Leptospirosis* tab has its three charts, the caveat, the peak-lag sentence matching `lepto_city_lag.csv` , and its controls work.
`CORE_FILES`	210-214	The outputs an older build already had.
`test_minimal_build_still_renders()`	217-227	With only those files (in a temporary folder via `BANTAY_APP_DATA`) the app still renders, without the optional tabs and with the four storms only.

`tests/test_e2e.py`

Drives the running dashboard in Chromium (pytest-playwright) and fails on any app exception or browser console error.

BLOCK	LINES	EXPLANATION
Setup	13-15	All tests here carry the `e2e` marker; the expected tabs.
`browser_context_args()`	18-20	A 1600 x 1300 window.
`app()`	23-32	Opens the dashboard, collects console and page errors, and asserts there were none after each test.
`idle()`, `wait_map()`, `no_exception()`, `main_title()`, `kpi()`	35-56	Helpers: wait for Streamlit to finish a run, wait for the map to draw, check for exceptions, read the title and a KPI.
`test_every_tab()`	59-68	Clicks every tab, screenshots it, and checks the open tab is in the URL.
`test_storm_and_shift()`	71-104	Moves the track with the keyboard, checks title, ferries and the upper-bound note, resets, switches to Tisoy and searches for a catalogue storm.
`test_dark_mode()`	107-114	In dark mode the map uses the dark "no data" colour.
`test_sus_submit()`	117-133	Fills and submits the usability form and checks the saved row.

`tests/capture.py`

Not a test: refreshes the dashboard figure used by the report and slides and records a silent walkthrough that follows the demo script.

BLOCK	LINES	EXPLANATION
Docstring, `FIGURE`, `VIDEO_DIR`	1-31	Usage (`--no-video`, `--no-shot`) and where the figure and video go.
`start()`	34-49	Starts the dashboard on a free port with a throw-away SUS file.
`ready()`, `centre()`, `unit_path()`, `glide()`	52-78	Wait for a run (and the map), the centre of an element, the screen position of a province found through the figure's own data, a slow mouse move.
`shot()`	81-91	Saves `docs/results/figures/dashboard.png` (Ompong, observed track, 1600 x 1300).
`video()`	94-159	Records the demo-script cues at 1920 x 1080 (map, tooltip, track shift, risk vs access, provinces, municipalities, forecast skill, validation) to `tests/artifacts/video/`.
`main()` and entry	162-178	Runs one or both.

7. DOCUMENTS

scripts/make_results.py

Draws every figure from the build outputs, writes the HTML for the report, slides, demo script and this guide, prints them to PDF with headless Chrome/Edge, and checks the report and slides for layout overflow. Every number in the text is computed from the outputs.

BLOCK	LINES	EXPLANATION
Paths and palette	38-44	Output folders and colours (teal, amber, red, grey; checked for colour-blind separation), and the teal map scale.
`_opt_csv()`, `_opt_pq()`	49-54	Read an optional output, or None.
`load()`	57-96	Reads every output the documents use (the optional ones may be missing).
`stats()`	99-172	Computes every number quoted in the documents (files, sources, coverage, scores, graph size, access, the storm table with both methods and vulnerable groups, top priorities, Sentinel-1 per storm and its EMS agreement), then the groups below, so the text can't drift from the data.

BLOCK	LINES	EXPLANATION
`_rng()`	175-176	"a" or "a-b" text for a range.
`stats_access()`	179-231	Network vs raster: the normal-day comparison, rank agreement per storm, the range over shifted tracks, ranking agreement of P; HRSL shares against the census; Sentinel-1 per municipality.
`census_age()`	234-241	Under-5 and 60+ shares of the 2020 census, to compare with HRSL.
`stats_calibration()`	244-254	The calibration summary and how many provinces tie on Ompong's top calibrated R (isotonic maps make ties).
`BENCH_LABEL` , `MODEL_LABEL`	257-260	Display names of the models.
`stats_models()`	263-286	The model comparison from `eval_models.csv` : which benchmarks are present or missing, the best model, median WIS, the variants' scores.
`median_wis_split()`	289-308	Re-scores the saved quantiles and benchmark files exactly like `forecast.evaluate_models()` : median WIS per model and the mean-WIS ratio to XGBoost below and above the 90th percentile of weekly cases.
`stats_catalog()`	311-341	One row per scored storm (catalogue + backtests): mean R, calibrated or not, people cut off by both methods, ferries, tiers; the skipped storms; rank agreement of the methods.
`_irr()`	344-348	The joint lag rows and the cumulative row of one leptospirosis analysis.
`stats_lepto()`	351-380	Leptospirosis numbers: city and national peaks, heavy-rain peak, the event-study rows, flooded city-storm pairs.
`stats_data()`	383-413	Data-finding numbers read from raw files (dengue units, 2016 case fatality, cases per year, how often the US gale radius is missing) and the usability responses, if any.
`s1_ems_agreement()`	416-446	Compares Sentinel-1 with Copernicus EMS inside each EMS area: area each mapped, overlap, precision and recall.
`style()` , `mono()` , `save()`	451-473	Registers the bundled fonts with matplotlib, sets a clean chart style, monospace tick labels, and saves PNGs at 220 dpi (fonts baked in).
`fig_timeline()`	478-494	National weekly dengue with the four typhoons and the 2020 collapse marked.
`fig_coverage()`	497-516	Share of panel cells with a value, per input.
`KIND_COLOR` , `fig_wis()`	519-569	Relative WIS of every model in the comparison (bars, with 1- and 4-week marks) and coverage by horizon against the 80-95% target.
`fig_reliability()`	572-589	R's reliability curve, raw and recalibrated.
`fig_methods()`	592-630	People cut off by the network and the raster: the observed tracks, then every proxy-flood track.
`fig_catalog()`	633-664	Mean R against people cut off for every scored storm, with the range between the two methods.
`fig_lepto()`	667-686	Leptospirosis IRR by lag, cities and national.
`fig_fan()`	689-708	Rolling-origin forecasts for Cagayan, 1 and 4 weeks ahead.
`_basemap()`	711-715	Grey province outlines and the Philippine extent for maps.
`fig_leadlag()`	718-739	The PCMCI+ links as arrows on the map (NCR districts share one label).
`fig_priority()`	742-762	Ompong's P map with track, swath and top-5 labels.
`fig_access_multiples()`	765-781	A per province for the four typhoons on one colour scale.
`fig_scatter()`	784-804	Ompong's risk vs access, marker shape = tier.
`fig_validation()`	807-823	Backtest correlations per storm.

BLOCK	LINES	EXPLANATION
`fig_s1()`	826-846	Sentinel-1 new-flood share per province for each storm (per-storm values, not the merged panel).
`fig_s1_ems()`	849-869	Sentinel-1 flood vs EMS outlines around Tuguegarao.
`fig_network()`	872-884	Ferry routes and hospitals over the islands (the access problem slide).
`fig_rolling()`	887-905	Diagram of the 8 rolling-origin refits (train on the past, score the next block).
`make_figures()`	908-913	Draws them all (a figure whose input is missing is None).
`FONT_CSS`	918-934	The bundled fonts (Big Shoulders Display, Barlow, Space Mono) and colour variables shared by all documents.
`REFERENCES`	936-972	The APA 7 reference list used by the report and slides.
`fmt_m()`, `storm_rows()`, `method_rows()`, `model_rows()`	975-1021	Millions text, and HTML table rows for the storm table, the two access methods and the model comparison.
`bench_missing_text()`	1024-1026	Names the expected benchmarks that are not in the build.
`catalog_rows()`, `s1_muni_rows()`	1029-1054	Rows for the storm catalogue (split over side-by-side tables) and for Sentinel-1 per municipality.
`eval_rows()`, `edge_rows()`, `sens_rows()`, `valid_rows()`, `source_rows()`	1057-1129	Rows for the evaluation, the graph links, the sensitivity, the backtest and the data sources.
`browser()`, `to_pdf()`	1134-1149	Finds Chrome or Edge and prints an HTML file to PDF headlessly.
`SCRIPT_CSS`, `_md_inline()`, `demo_script_html()`	1152-1205	Turns the demo-script Markdown (the editable source) into a styled landscape PDF.
`GUIDE_CSS`, `code_guide_html()`	1208-1253	Turns this guide (`docs/CODE_GUIDE.md`) into a styled A4 PDF: repo file links become plain text, the contents links stay, and the Block / Lines tables get a line-number column style.
`OVERFLOW_JS`, `check_overflow()`	1256-1298	Lays each page out in headless Chrome and lists any element that runs into the footer band or off the edge, so overflow is caught automatically.
`main()` and entry	1301-1333	Loads, computes, draws, writes the HTML, checks the report and slides for overflow, prints the PDFs (report, slides, code guide, demo script). `--no-pdf` skips printing.

scripts/results_pages.py (EVALUATION REPORT, A4)

BLOCK	LINES	EXPLANATION
`REPORT_CSS`	19-64	Page layout: A4 pages with a dark side rail (page number and section), condensed headings, tables, notes, the cover.
`_page()`	67-69	Wraps one page's content with its rail and footer.
`_lst()`	72-74	"a, b and c" text.
`_targets()`	77-115	The scorecard rows: each proposal objective, the result and met / partly / not yet, decided by which outputs are present.
`d_min()`, `d_max()`	118-123	Smallest and largest value of a validation column.
`report_html()`	126-543	All pages in order: cover, summary, data, panel, methods, forecast, model comparison, calibration, graph, Sentinel-1, access, the two methods, Ompong priority, the catalogue, validation, leptospirosis, scorecard, limitations, references. Pages whose outputs are missing are left out and the text switches; page cross-references are filled in from the page labels, and figures are numbered in page order at the end.

scripts/results_slides.py (20 SLIDES, 16:9)

BLOCK	LINES	EXPLANATION
`SLIDE_CSS`	16-89	Slide layout: dark slate background, amber signal tags, white chart cards, the pipeline diagram (grid with arrows that touch the boxes), timelines, stat blocks.
`N`	91	Slide count (checked at the end).
`_slide()`	94-97	Wraps a slide: signal tag numbered with the slide (Signal 01-20), the judging criterion it answers, brand and page number.
`_img()`	100-101	A figure on a white card at a set height.
`slides_html()`	104-366	The 20 slides: title with headline results; the problem (typhoons and disease, the cascade, access in an archipelago); design; data; data challenges; method; nine result slides (forecast skill and the models, calibrated surge risk, outbreak links, Sentinel-1 flood, access loss by two methods, the Ompong case, the storm catalogue, validation, leptospirosis); the demo; deployment in the loop; sources. No names or internal file names (anonymous judging).

scripts/make_proposal.py

Renders the revised concept proposal (`docs/proposal/Bantay-Baha_Proposal_v3.html` , the editable source) to PDF, with automatic word counts. `--no-pdf` prints the counts only.

BLOCK	LINES	EXPLANATION
`ROOT` , `SRC` , `PDF`	20-22	The HTML source and the PDF next to it.
`browser()`	25-30	Finds Chrome or Edge.
`words()`	33-36	Word count of an HTML fragment, headings excluded.
`fill_counts()`	39-59	Fills every word-count placeholder with the count of the section it names (and its limit), and the reference placeholder with the number of references.
`main()`	62-81	Prints the counts, stops if a section is over its limit (the executive summary: 250 words), then prints the filled HTML to PDF through a temporary file.
entry	84-87	`--no-pdf` .

scripts/check_code_guide.py

Keeps this guide honest: `python scripts/check_code_guide.py` prints every link or block that no longer matches the code and exits 1, or prints "all links and blocks match the code". Three rules follow from it when you edit the guide:

- every line link `[a-b](../path#La-Lb)` must point inside an existing file;
- in a Block | Lines | Explanation table, a first cell that names exactly one function, method or class (`` `fetch()` `` , `` `AccessModel` `` , `` `FrictionGrid.solve()` ``) must give its exact span, from the `def` / `class` line (or its first decorator) to its last line;
- a heading that names a Python file starts that file's section, and every top-level function and class of the file must be mentioned (in backticks) somewhere in its section.

BLOCK	LINES	EXPLANATION
`ROOT` ... `NAME`	19-23	The guide's path and the three patterns: a line link, a section heading, a function-name cell.
`spans()`	26-42	Every function, method and class of a file with its first line (decorators included) and last line, from Python's `ast` , plus the set of top-level names.
`main()`	45-89	Reads the guide line by line, tracking the current section and the names it mentions; checks each link and each function row; then lists the top-level names a section never mentions and sections for files that don't exist.
entry	92-93	Exit code 1 when there are problems.

8. BENCHMARK MODELS

The proposal compares XGBoost with three outside models: **Chronos-2** and **TabPFN-TS** (zero-shot foundation models) and **MPNN-LSTM** (a graph neural network). They are optional and run outside the build:

- **A separate venv.** They need torch with CUDA, and their packages pin an older pandas than the main venv, so they live in `%USERPROFILE%\.venvs\bantay-bench` (Python 3.12). `requirements-bench.txt` holds the exact install commands and the tested versions. Model weights download into the user cache on first run; no token or account is needed.
- **The output contract** (`benchmark_common.py`). Each model writes `data/processed/app/bench_<model>.parquet` : one row per (unit, forecast origin, horizon 1-4) of the XGBoost evaluation (exactly the rows of `eval_scores.parquet`), with `q` = the 23 FluSight quantiles of log1p(incidence per 100k), ascending; and `bench_<model>.json` with versions, device, runtime, inputs and notes. Every input at an origin must come from weeks up to that origin.
- **Then** `benchmark_score.py` writes `bench_summary.csv` (headline models and variants, with median WIS), and `build.bat ensemble` re-scores everything and builds the median ensemble without refitting XGBoost.
- **The** `forecast.BENCH_MODELS` **rule.** Only `chronos2` , `tabpfn_ts` and `mpnn_lstm` enter the model table and the median ensemble. Every other `bench_*` file (`chronos2_log1p` , `mpnn_lstm_border` , ...) is a variant: scored by `benchmark_score.py` , listed on the dashboard as "not used", ignored by the build so it cannot tilt the ensemble. To make a new model a headline benchmark, add its name to `BENCH_MODELS` .

The storm scenarios always use XGBoost: the benchmarks take no storm-track input, so they can't score a shifted track.

scripts/benchmark_common.py

Shared helpers; not a script. Needs only pandas, numpy and `bantay` .

BLOCK	LINES	EXPLANATION
Docstring	1-11	The output contract (see above).
Path setup, `Q` , `HORIZONS` , `KEY`	19-29	Puts the repo on the import path; the 23 levels, horizons 1-4 and the row key.
`eval_rows()`	32-38	The rolling-origin evaluation rows (unit, origin, horizon) with observed cases and the per-row XGBoost and naive scores, from `eval_scores.parquet` .
`panel()`	41-48	The build's panel with y = log1p(incidence), sorted by unit and week.
`wide()`	51-53	A unit x week matrix of one column.
`interp_levels()`	56-62	Maps quantiles at a model's own levels onto the 23 FluSight levels by linear interpolation in level space.
`write_bench()`	65-88	Checks the contract (exactly the evaluation rows, 23 finite sorted quantiles, Sunday origins) and writes the parquet and json.
`versions()` , `device_info()`	91-108	Package versions and the GPU used, for the json.
`quick_score()`	111-122	Prints a model's WIS and its ratio to naive and XGBoost on the same rows, right after a run.

scripts/benchmark_tsfm.py (CHRONOS-2)

BLOCK	LINES	EXPLANATION
Docstring	1-27	The setup, the context, the variants and why the headline feeds incidence (see `main()`).
`COVARIATES`	37-38	Past-only covariates: CHIRPS rain, river rain and its excess, gale hit, nearby wind, storm x hazard, Sentinel-1 flood (missing values passed as missing, never 0).

BLOCK	LINES	EXPLANATION
`main()`	41-102	Zero-shot Chronos-2 at every evaluation origin: each province's history up to the origin (with the covariates unless `--variant nocov`), 4 weeks ahead, its 21 quantile levels mapped to the 23. The headline (`chronos2`) feeds incidence and maps the output to log1p; feeding log1p directly (`--space log1p`, the variant `chronos2_log1p`) gave huge upper tails after reporting spikes. No training, so no leakage.
entry	105-113	`--device cuda`, `--variant cov/nocov`, `--cross-learning`, `--batch-size`, `--space incidence/log1p`.

scripts/benchmark_tabpfn.py (TABPFN-TS)

BLOCK	LINES	EXPLANATION
Docstring, telemetry off	1-28	Why the ungated TabPFN v2 weights are pinned (the package default v3.5 is gated) and the variants; telemetry is switched off.
`LAG`	35	Covariates are lagged 4 weeks, so their values for the 4 target weeks are known at the origin.
`lagged_covariates()`	38-46	Rain (7-day and 4-week), river rain and recent gale hit, shifted by `LAG` (the `lagcov` variant).
`main()`	49-106	TabPFN-TS run locally (GPU when available) at every origin on each province's log1p incidence up to the origin (its own calendar features; covariates only in `lagcov`), asking for the 23 levels directly. Each origin is cached in `data/interim/bench/tabpfn_<variant>/`, so an interrupted run resumes.
entry	109-113	`--variant uni/lagcov`.

scripts/benchmark_mpnn.py (MPNN-LSTM)

BLOCK	LINES	EXPLANATION
Docstring	1-30	The graph, the model and the leak-free training protocol in full.
Constants	42-46	12-week input window, 500 m border buffer, three ferry-name aliases, the node features and the ones that are z-scored.
`_norm()`	49-50	Name normaliser for matching RoRo province names to units.
`adjacency()`	53-99	The fixed graph: units that share a border (polygons buffered by 500 m), plus units joined by a RoRo route (a 2024-2026 proxy), or none (`--adjacency none`, the graph ablation). Reports edge counts and how well route names match the port coordinates.
`norm_adj()`	102-105	The usual normalised adjacency D^-1/2 (A + I) D^-1/2.
`arrays()`	108-120	Unit x week arrays of the inputs; y forward-filled (past only) with a missing flag; week-of-year terms; log population.
`build_tensors()`	123-143	Scales y per unit and rain per feature with statistics from weeks up to the refit date only.
`make_net()`	146-174	The network: two graph-convolution layers per week, an LSTM over the window per province, and a head that outputs 4 horizons x 23 quantiles that cannot cross (a base plus positive steps), as a change from the last value.
`samples()`	177-185	Input windows, last year's values for the target weeks (known at the origin) and targets for a set of origins.
`pinball()`	188-193	Quantile (pinball) loss averaged over levels and horizons, ignoring missing targets.
`train()`	196-223	Adam on mini-batches of weekly snapshots; with a validation set, early stopping returns the best epoch.

BLOCK	LINES	EXPLANATION
`main()`	226-299	The rolling origin of the XGBoost test: at each 13-week refit, training targets end at the refit date, epochs are chosen on the last 13 training origins, the model is refit on all of them, and the next block is forecast; quantiles are averaged over the seeds (3 by default).
entry	302-309	`--adjacency border_ferry/border/none`, `--device`, `--seeds`, `--max-epochs`.

scripts/benchmark_score.py

Scores the benchmark files exactly like the XGBoost evaluation; runs in either venv.

BLOCK	LINES	EXPLANATION
`MODELS`	25	The three headline models, scored first.
`per_row()`	28-35	One model's per-row WIS and 50%/90% coverage on case counts, on the evaluation rows.
`agg()`	38-50	Mean WIS and its ratio to naive and XGBoost, coverage, median WIS, the ratio without the model's single worst origin, and the share of WIS in the top 0.1% of rows (a mean on counts can be dominated by a few rows).
`main()`	53-71	Scores every requested model on the rows where both reference scores exist and writes `bench_summary.csv` (per horizon and all).
`all_models()`	74-76	The headline models found, then every other `bench_*` file.
entry	79-80	Model names are optional arguments.

9. THE EDA NOTEBOOK

`notebooks/00_data_eda.ipynb` checks each raw dataset before any modelling: does it exist, what does it cover, and does it match what the proposal assumed. Findings are collected with `note(dataset, msg)` and shown in one table at the end.

CELL	SECTION	WHAT IT CHECKS
0-1	Setup	Dataset list; imports, paths, the `note()` and `have()` helpers, a summary of `MANIFEST.csv` by source.
2-4	1. DOH dengue	Rows, weeks (all Sundays), 126 units incl. cities, cases and deaths per year (2016 deaths implausible, 2020 collapse), national curve and missing-unit share.
5-7	2. CCHAIN	City-week disease totals; leptospirosis series for 12 cities, 2008-2022.
8-9	3. Mendeley	National weekly diseases 2019-2023; coverage per year.
10-11	4. OpenDengue	Philippine series by space/time resolution; correlation with the DOH national total.
12-15	5. Boundaries, census	COD-AB layers and versions; the name match of the 126 dengue units (the origin of the crosswalk logic); census sheets.
16-17	6. IBTrACS	The four storms, fixes, max wind, how often the 34-kt radius is present; track map.
18-19	7. Rainfall	CHIRPS days, missing days, julian time axis; IMERG files.
20-23	8. Flood	EMS packages and EMSR312 flood area; NOAH files and which provinces have maps; GFD and Sentinel-1 inventory.
24-25	9. HydroRIVERS	Reaches touching the Philippines and outlets.
26-29	10. Access	roroph routes and coordinate coarseness; OSM roads and ferries; health facilities; friction raster.
30-31	11. Population	WorldPop total and Meta HRSL layers.
32-33	Summary	All findings in one table.

10. EVERY FILE THE PIPELINE WRITES

All build outputs under `data/` stay out of git (only `data/raw/MANIFEST.csv` and `data/crosswalks/` are committed).

FILE	WRITTEN BY	READ BY
`data/raw/**`, `data/raw/MANIFEST.csv`	download scripts, `sync_data.py`	everything
`data/crosswalks/adm2_to_unit.csv`, `dengue_loc_to_unit.csv`	`units.build_crosswalks` (then reviewed)	all later stages
`data/processed/units.gpkg`	`units.build_units_geometry` (`crosswalk`)	every stage, maps
`data/processed/status.json`	`config.record` (every stage)	build summary, data card, dashboard, results
`data/interim/units_100m.tif`, `noah_5yr_hazard_100m.tif`, `noah_25yr_hazard_100m.tif`	`grids.build_unit_raster`, `flood.build_hazard_raster` (`panel`)	hazard shares, access, Sentinel-1
`data/interim/rain_chirps_unit_week.parquet`, `rain_imerg_unit_week.parquet`, `storm_unit_week.parquet`	`panel` (cached)	`panel`
`data/processed/panel.parquet`, `panel_card.md`	`panel.build_panel`	leadlag, forecast, scenarios, catalog, lepto, benchmarks, results
`data/processed/s1_unit_storm.parquet`	`panel.s1_layers`	results
`data/processed/app/leadlag_edges_pcmciplus.parquet`, `leadlag_nodes_pcmciplus.parquet`, `leadlag_edges_xcorr.parquet`, `leadlag_nodes_xcorr.parquet`, `leadlag_meta.json`	`leadlag` stage	dashboard, scenarios, catalog, results
`data/processed/app/eval_summary.csv`, `eval_forecasts.parquet`	`forecast` stage	dashboard, results
`data/processed/app/eval_scores.parquet`	`forecast` stage	benchmark scripts (it fixes their rows)
`data/processed/app/eval_graphs.csv`	`forecast` stage	results
`data/processed/app/surge_reliability.csv`, `surge_calibration.json`	`forecast` stage	dashboard, results
`data/processed/app/eval_cum4.parquet`	`forecast` stage	scenarios, catalog (`forecast.surge_calibrator`)
`data/processed/app/eval_quantiles.parquet`	`forecast` stage	`ensemble` stage, results
`data/processed/app/bench_<model>.parquet`, `bench_<model>.json`	`scripts/benchmark_*.py`	`ensemble` stage, `benchmark_score.py`, results
`data/processed/app/bench_summary.csv`	`benchmark_score.py`	dashboard, results
`data/interim/bench/tabpfn_<variant>/`	`benchmark_tabpfn.py` (per-origin cache)	`benchmark_tabpfn.py`
`data/processed/app/eval_models.csv`	`ensemble` stage	dashboard, results
`data/processed/app/eval_models.json`, `eval_forecasts_models.parquet`	`ensemble` stage	results; dashboard (benchmark fan charts)

FILE	WRITTEN BY	READ BY
`data/interim/road_graph.npz`, `pop_cells_1km.parquet`, `pop_cells_hrsl.parquet`, `adm3_100m.tif`	`access`, `grids` (`scenarios`, `catalog`)	scenarios, catalog
`data/processed/app/scenarios.parquet`, `scenarios_adm3.parquet`, `tracks.parquet`, `swaths.parquet`, `ferries_status.parquet`, `ferries.parquet`, `hospitals.parquet`	`scenarios` stage	dashboard, results
`data/processed/app/access_methods.csv`	`scenarios` stage	dashboard, results
`data/processed/app/s1_adm3.parquet`	`scenarios` stage (`flood.s1_adm3_flood`)	dashboard, results
`data/processed/app/validation.csv`, `sensitivity.csv`, `benchmark_ncr.csv`	`scenarios` stage	dashboard, results
`data/processed/app/catalog_storms.csv`	`catalog` stage	dashboard, results
`data/processed/app/scenarios_catalog.parquet`, `scenarios_catalog_adm3.parquet`, `tracks_catalog.parquet`, `swaths_catalog.parquet`, `ferries_status_catalog.parquet`, `access_methods_catalog.csv`	`catalog` stage	dashboard, results
`data/processed/app/units_simplified.geojson`	`app` stage	dashboard, results
`data/processed/app/lepto_alignment.csv`, `lepto_national_lag.csv`, `lepto_city_lag.csv`, `lepto_events.csv`, `lepto_events_did.csv`, `lepto_events_summary.csv`, `lepto_events_ratio.csv`, `lepto_weekly.parquet`	`lepto` stage	dashboard, results
`data/processed/lepto_summary.md`, `data/interim/lepto_city_chirps_week.parquet`	`lepto` stage	people; `lepto` (cache)
`data/processed/sus/sus_responses.csv`	dashboard *Usability test* tab (`sus.save`)	dashboard, results
`docs/results/*.pdf`, `docs/results/figures/f*.png`, `docs/results/src/*.html`	`make_results.py`	people
`docs/results/figures/dashboard.png`, `tests/artifacts/`	`tests/capture.py`, the tests	report and slides; people
`docs/proposal/Bantay-Baha_Proposal_v3.pdf`	`make_proposal.py`	people

11. GLOSSARY

TERM	MEANING
R	Surge probability: chance the next 4 weeks' incidence exceeds the province's 90th percentile.
Calibrated R / R raw	The model's raw probability (`R_raw`) mapped by an isotonic fit to how often surges really happened, using past forecasts only. The calibrated value is `R`, used for P and the tiers.
A	Access loss: share of a province's people who lose their route to a hospital in the storm.
Network / raster method	Two ways to compute travel time: the road and ferry graph (main) and the MAP friction surface (check). `A_lo` / `A_hi` (and `P_lo` / `P_hi`) span the two.
P	Priority = R x A.

TERM	MEANING
Tier	The provisional action rule: deploy mobile clinics, pre-position supplies, or monitor.
Unit	One of the 85 harmonised 2016-2020 provinces (NCR is split into its 4 districts).
Incidence	Cases per 100,000 people.
Rolling origin	Testing a forecaster as if in real time: train on the past, forecast forward, move the cut-off forward.
WIS / relative WIS	Weighted interval score of a forecast range (lower is better); relative WIS < 1 means better than the seasonal-naive baseline.
Seasonal naive	"Same week last year", the baseline forecast.
Ablation	The same model with a group of features removed (no graph, no rain/river/storm/flood), to see what that group adds.
Benchmark / median ensemble	Outside models scored on the same rows (Chronos-2, TabPFN-TS, MPNN-LSTM); the ensemble is the median of their and XGBoost's quantiles.
Quantile	A forecast at a probability level; 23 of them describe the full range.
Coverage	How often the truth falls inside a stated range (90% range should catch ~90%).
Brier score	Error of a probability forecast (lower is better).
PCMCI+	A causal-discovery algorithm for time series that keeps a link only if it survives conditioning on other series and their pasts.
FDR / Benjamini-Hochberg	A correction that controls the share of false links when testing thousands of pairs.
Sentinel / PageRank / Louvain	A province others follow (scored with PageRank); groups of provinces that move together (Louvain).
Gale swath	Area that saw gale-force (34-knot) winds along the track.
Shifted track	The same storm moved north/south/east/west, for "what if" scenarios.
Storm catalogue	Every other 2016-2020 storm whose gale swath reached land, scored on its observed track with the NOAH proxy.
NOAH hazard proxy	Using flood-hazard maps instead of observed flood to decide which roads close; an upper bound.
Sentinel-1	Radar satellite that sees flood through clouds; our observed-flood source (a lower bound).
EMS	Copernicus Emergency Management Service flood maps; used to check Sentinel-1.
HRSL	Meta's High Resolution Settlement Layer: counts of children under 5 and people aged 60+, used for the vulnerable people cut off.
Dijkstra	Shortest-path algorithm; run from all hospitals at once to get each place's nearest-hospital time.
IRR / distributed lag	Incidence rate ratio (1 = no change); a distributed-lag model fits several weeks of lagged rain together.
Event study / placebo	Comparing cities inside and outside a storm's swath around landfall; the weeks before landfall should show no difference.
SUS	System Usability Scale: ten statements scored 0-100; about 68 is average.
Crosswalk	A table mapping one set of place codes or names onto another.
Manifest	`data/raw/MANIFEST.csv` : every raw file with size and checksum.